Birzeit University
Faculty of Engineering and Technology
Department of Computer Science

(COMP338) – Artificial Intelligence

Course Project 2: Classify News Headlines into Categories Using Decision Tree and Logistic Regression

Prepared By:

Ayman Mohammad Jamal – 1220115
Nagham Kamal Khawaja – 1220825

Instructors:

Dr. Abdelrahman Hamarsha
Dr. Mohammed Helal

Birzeit University
January – 2026

Abstract

The purpose of this project is to create an 'automated' system that can classify news headlines using machine learning techniques based on four main categories: World, Sports, Business, and Science/Technology. In order to accomplish this goal, we researched and applied two popular algorithms used in text classification: Logistic Regression and Decision Trees.

The first step in the project was to pre-process the news articles' headline data in a way that removes unnecessary or unneeded data elements, as well as preparing the headline text data for analysis. The processed article data was then represented numerically by converting it into numeric feature vectors using the TF-IDF method; therefore, each article's vector would contain the words in that particular article and their corresponding weightings, which established how important each word was in regards to the total weight of the vector. Both Logistic Regression and Decision Tree algorithms were trained with this processed data and evaluated with a number of different measures of performance, including accuracy, precision, recall, and f1 score.

Table of Contents

1. **Introduction** 1

1.1. Overview of Target News Categories 2

1.2. Example of TF-IDF Word Representation 2

2. **Dataset Overview** 3

2.1. Dataset Structure 3

2.2. Dataset Size 4

2.3. Why AG News? 4

3. **Machine Learning Pipeline** 5

3.1. Load Data 5

3.2. Split Data 6

3.3. Data Preprocessing 7

3.3.1. Lowercasing 7

3.3.2. Removing Punctuation and Special Characters 7

3.3.3. Remove extra whitespace 7

3.3.4. Tokenization 8

3.3.5. Stopwords Removal 8

.3.3.6 Filter by length 8

3.4. Build vocabulary 9

3.5. Feature Extraction 9

3.5.1. TF-IDF (Term Frequency - Inverse Document Frequency) 9

3.5.2. The TF-IDF used in this project: 10

3.5.3. TF-IDF Parameters 10

3.5.4. Vocabulary Construction 11

3.5.5. Example of TF-IDF Output 11

.4 **Algorithms Used** 13

.4.1 Logistic Regression 13

4.1.1. Advantages and Disadvantages of Logistic Regression 14

4.1.2. Logistic Regression Workflow 15

4.2. Decision Tree 17

4.2.1. Advantages and Disadvantages of Decision Trees 17

4.2.2. Internal Working and Splitting Criterion 17

4.2.3. Decision Tree Classification Workflow 18

5. **Implementation Details** 20

5.1. Loading 20
5.2. Preparing Dataset 23
.5.3 Preprocessing Code 23
.5.4 TF-IDF Code 23
.5.5 Training Logistic Regression 23
5.6. Training Decision Tree 25
.6 **Evaluation Metrics** 26
6.1. Accuracy 26
6.2. Precision 26
6.3. Recall 27
6.4. F1-score 27
6.5. Confusion Matrix (Supporting Analysis) 28
7. **Experimental Results** 29
.7.1 Interpretation of Results 32
7.1.1. One-Vs-Rest (OVR) 33
.7.2 Confusion Matrix Interpretation 34
.7.2.1 Decision Tree– Confusion Matrix Interpretation 34
7.2.2. Logistic Regression – Confusion Matrix Interpretation 35
.7.3 Performance Comparison of the Two Classification Models 37
8. **Graphs** 38
9. **Conclusion** 39
10. **Feedback** 40
11. **References** 41

Acronyms and Abbreviations

Acronym	Full Term	Description
AI	Artificial-Intelligence	Field focused on building systems that can learn and make decisions.
ML	Machine-Learning	Subfield of AI that enables systems to learn from data.
NLP	Natura-Language-Processing	AI branch that deals with understanding and processing human language.
TF-IDF	Term-Frequency-Inverse-Document Frequency	Technique to convert text into numerical features.
LR	Logistic -Regression	A supervised learning algorithm used for classification tasks.
DT	Decision -Tree	Atree-based supervised learning algorithm used for classification and regression.
NLP	Natural-Language-Processing	Language understanding and processing techniques.
Acc	Accuracy	Metric measuring the percentage of correct predictions.
Prec	Precision	Metric measuring how many predicted positives were actually correct.
Rec	Recall	Metric measuring how many actual positives were identified correctly.
F1	F1-Score	Harmonic mean of Precision and Recall.
CSV	Comma-Separated Values	Common file format for datasets.
API	Application-Programming Interface	Interface for software communication (if referenced).
CPU	Central -Processing -Unit	Processor used for computation (optional but common).
BZU	Birzeit University	The university name abbreviation.

List of Figures

Figure 1-1:Example of News Headline Classification Pipeline [1] 1
Figure 1-2:TF-IDF Word Representation [2] 2
Figure 3-1:split_data 6
Figure 3-2: Lowercasing 7
Figure 3-3 :Removing Punctuation and Special Characters 7
Figure 3-4:Removing Punctuation and Special Characters 7
Figure 3-5:Common English stopwords 8
Figure 3-6:Tokenization&Filter&Remove stopwords 8
Figure 3-7:build_vocabulary 9
Figure 3-8:F-IDF Parameters [3] 10
Figure 3-9:Compute normalized term-frequency vectors for texts using existing vocabulary 12
Figure 3-10:Fit IDF on training set and return TF-IDF for training 12
Figure 3-11:Transform texts to TF-IDF using already-fitted IDF 13
Figure 4-1:logistic regression mathematically expressed [4] 13
Figure 4-2:descent rule 16
Figure 4-3 :Logistic Regression Workflow [6] 16
Figure 4-4:Decision Tree Workflow 19
Figure 5-1: Imports & Global Settings 20
Figure 5-2:HTTP Session Setup 20
Figure 5-3:Robust JSON Request 21
Figure 5-4:Download One Split 21
Figure 5-5:Main Download Logic 22
Figure 5-6:Training Logistic -1 23
Figure 5-7:Training Logistic -2 23
Figure 5-8:Train one-vs-rest classifiers 24
Figure 5-9:predict LogisticRegression 24
Figure 5-10:build DecisionTreeGenerating metrics 25
Figure 5-11 :DecisionTree fit&predic 25
Figure 6-1:Accuracy 26
Figure 6-2:Precision 26
Figure 6-3:Recall 27
Figure 6-4:F1 27
Figure 6-5:confusion_matrix 28
Figure 7-1:INTERPRETATION 32
Figure 7-2:Evaluating Logistic Regression 32
Figure 7-3:Evaluating Decision Tree 32
Figure 7-4:Metrics Comparison 33
Figure 7-5 :decision tree 34
Figure 7-6 :Logistic Regression 35
Figure 8-7-1:Decision Tree 38
Figure 9-1: Conclusion 39

List of Tables

Table 1-1:Target News Categories 2
Table 4-1: Advantages and Disadvantages 14
Table 4-2:Advantages and Disadvantages 17

1. Introduction

Natural Language Processing (NLP) utilizes many of the tools available through modern Machine Learning (ML), including text classification, to assist with the rapid growth and evolution of the internet and digital information collections in terms of the sheer volume of textual information being generated daily. Internet news sites and social media sites have developed methods to utilize automated process to quickly categorize the content published and better serve the user with relevant search results.

As the total number of news stories, headlines, etc. continue to grow (and grow exponentially), there is an ongoing need for machine learning solutions that are both readily accessible and can successfully identify the relevant context in short text messages.

Currently, machine learning techniques have been used to assist people in this area of difficulty by helping them identify patterns in the text without the need for a human to manually search for the right pattern. In this project, we will evaluate two common algorithms used for supervised learning to classify news headlines into four categories: World, Sports, Business, and Science/Technology. The overall workflow for the project will include Data Preprocessing, Text Cleaning, TF-IDF Vectorization for feature extraction and Model Training, Performance Evaluation, and Visualization of the Results.

Since many times one or both of the algorithms will be proven inferior or superior to the other in terms of overall performance and will contain both the strengths and limitations relative to the data being classified, it is important to compare the results of both algorithms through established performance metrics such as Accuracy, Precision, Recall and F1 Score [1].

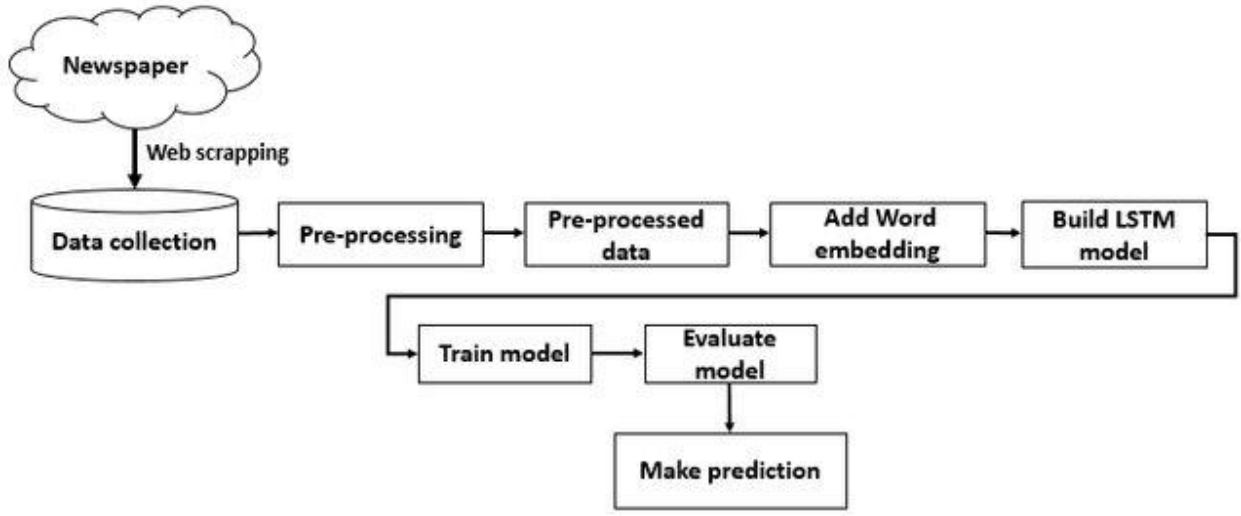

Figure 1-1:Example of News Headline Classification Pipeline [1]

This pipeline illustrates the full workflow used in the project, starting from raw text preprocessing and TF-IDF feature extraction to model training and prediction.

1.1. Overview of Target News Categories

Table 1-1:Target News Categories

Category	Code	Description
World	0	International news and global affairs
Sports	1	Headlines related to sports events and athletes
Business	2	Economic news, markets, and financial updates
Science/Tech	3	Technological innovations, research, and scientific topics

1.2. Example of TF-IDF Word Representation

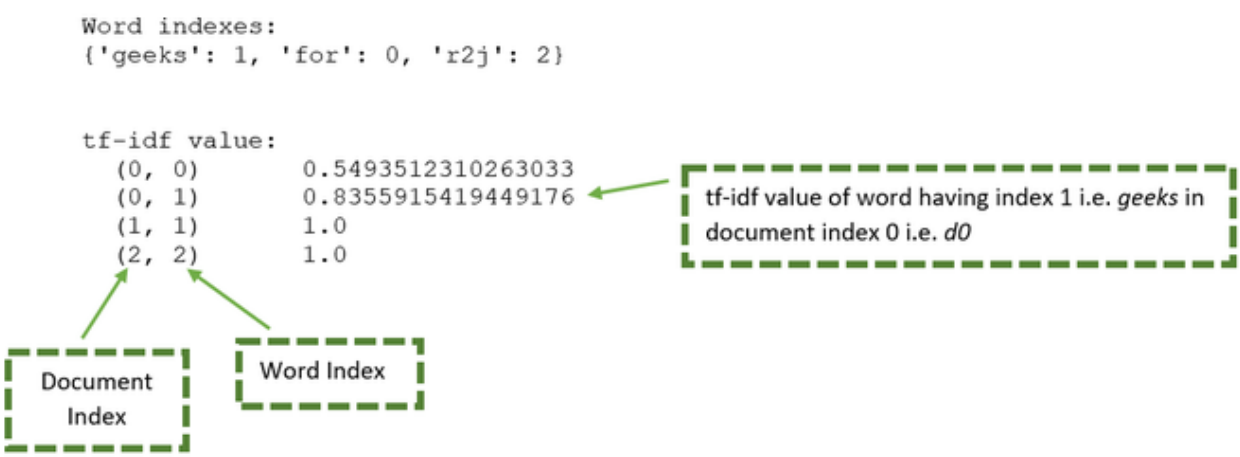

Figure 1-2:TF-IDF Word Representation [2]

TF-IDF transforms raw text into numerical vectors by weighing the importance of words across all documents, enabling machine learning algorithms to understand textual input.

2. Dataset Overview

AG News corpus serves as the underlying dataset for our project; it is widely used as an example of a text classification benchmark dataset. In addition to its utility in testing machine learning models with respect to text classification tasks, this dataset includes both news headlines and their corresponding categorical labels. As such, the AG News corpus is particularly useful in evaluating machine learning models on real-world applications related to text classification.

The dataset is publicly available on HuggingFace under the repository:

→wangrongsheng/ag_news · Datasets at Hugging Face

AG News consists of **four primary categories**, each representing a major news domain:

- **World (0)** – International news and global political developments
- **Sports (1)** – Sports events, competitions, players, and match reports
- **Business (2)** – Financial markets, economic trends, and corporate activities
- **Science/Technology (3)** – Scientific discoveries, innovations, and tech industry news

In this project, only the **headline text** (short title) is used as the input feature, while the **label** represents the target category.

2.1. Dataset Structure

Each record in the dataset includes:

- **text**: a short news headline
- **label**: an integer representing one of the four categories

An example data entry is shown below:

"Wall St. Bears Claw Back Into the Black (Reuters) Reuters - Short-sellers, Wall Street's dwindling band of ultra-cynics, are seeing green again.", 2

In this example:

- **Headlines**:
  "Wall St. Bears Claw Back Into the Black (Reuters) Reuters - Short-sellers, Wall Street's dwindling band of ultra-cynics, are seeing green again."
- **label**:

  2 =Business

This headline discusses financial market movement, which corresponds to the Business category.

2.2. Dataset Size

Depending on the version used, typical AG News splits include:

Split	Number of Samples
Training Set	102080
Test Set	25520

This large number of samples makes the dataset suitable for training traditional machine learning models such as Logistic Regression and Decision Trees.

2.3. Why AG News?

AG News is chosen because:

- It covers diverse real-world topics
- Headlines are short, challenging, and ideal for testing NLP models
- Balanced classes: suitable for fair evaluation
- Popular in research: results can be compared with known baselines

Overall, the AG News dataset provides an excellent foundation for training, testing, and comparing text classification models.

3. Machine Learning Pipeline

The ML pipeline outlines the workflow from converting raw text-based data into useful insight prediction. The pipeline will ensure that the ML pipeline has cleaned, normalized and transformed the raw text dataset into a numerical representation for training/validation of classification ML models. In defining the process into clear, systematic steps, the ML pipeline provides consistency, repeatability and the ability to efficiently deal with the large dataset that usually includes text-based data, such as AG News.

For this project, the ML pipeline is structured with three main components

(1) Data PreProcessing,

(2) Feature Extraction and

(3) Model Training and Evaluation (to be covered later in this report)

The following sections describe the first two components of the pipeline in detail:

Data PreProcessing and Feature Extraction.

3.1. Load Data

The load_data function loads the ag news dataset, which provides all of the necessary information about the dataset so that it can be formatted properly and organised structurally to be used later in the machine learning workflow.

This project downloaded the original dataset from the huggingface website as csv and saved it to your local machine. The load_data function opens the downloaded csv file and extracts the main feature of the data, which is the text of the news headline, as well as the associated label for that record.

By loading the dataset from a csv file, the load_data function will enable future uses of this data to be much quicker than by using the huggingface website or other internet-based api's.

The load_data function will return the text (headlines) and the labels in a consistent format for use during all of the next steps of producing the final dataset including data preprocessing, feature extraction, and model training.

By placing the load_data functionality within its own dedicated function, the load_data function will also allow developers to write reusable modules of code that will not only enable them to maintain the same source of information for the ag news dataset but will be able to be updated in the future with little to no additional development.

3.2. Split Data

split_data is responsible for dividing the full dataset into two distinct subsets: a training set and a testing set. The method receives the complete collection of texts and labels as its input and applies a predefined split ratio (e.g., 80% training and 20% testing). This produces two separate partitions:

- X_train / y_train – used exclusively during the training phase
- X_test / y_test – used only during evaluation

The primary purpose of this step is to ensure that the model is evaluated on data it has never seen before. This separation is critical for measuring the model's generalization ability, since performance on unseen data is a far more reliable indicator of real-world behavior.

In this project, sampling is applied only to the training split, not to the testing split. This is intentional: the training set may be reduced (limited to 5000 samples) to decrease computational cost, but the testing set must remain untouched so that the evaluation remains statistically meaningful, unbiased, and representative of the original dataset distribution.

By encapsulating this logic inside a dedicated function, the code becomes more modular and flexible, making it easy to modify the splitting strategy (different test sizes, random seeds) without affecting other components of the pipeline. This clean separation between data preparation and model training helps maintain reproducibility, readability, and correctness.

```python
class TextClassifier:  4 usages
    def split_data(self, data, test_split=0.2, random_seed=42):  1 usage
        if not (0 < test_split < 1):
            raise ValueError(f"test_split must be between 0 and 1 (got {test_split}).")


        random.seed(random_seed)
        random.shuffle(data)

        n = len(data)
        test_size = int(n * test_split)


        test_size = max(1, min(test_size, n - 1))
        split_idx = n - test_size

        train_data = data[:split_idx]
        test_data = data[split_idx:]


        max_train_samples = self.config["data"].get("max_train_samples", None)
        if max_train_samples is not None and len(train_data) > max_train_samples:
            random.seed(random_seed)
            train_data = random.sample(train_data, max_train_samples)
            print(f"Train samples after limit: {len(train_data)}, Test samples: {len(test_data)}")
        else:
            print(f"Train samples: {len(train_data)}, Test samples: {len(test_data)}")

        return train_data, test_data
```

Figure 3-1:split_data

3.3. Data Preprocessing

Raw text data, especially news headlines, has much noise and inconsistency due to the presence of punctuation symbols, special characters, digits, and different letter case combinations. Therefore, prior to applying any of the machine learning models on the data set, Pre-processing steps will need to be completed in order to clean and standardize the content of the data set, so the features (text) will have higher quality and more stable performance from the model.

The steps used in pre-processing for this project are as follows:

3.3.1. Lowercasing

All text is converted to lowercase to avoid treating words like “Market” and “market” as different tokens.

```
def clean_text(self, text):  1 usage
    """Clean and normalize text"""
    # Convert to lowercase
    if self.config.get('lowercase', True):
        text = text.lower()
```

Figure 3-2: Lowercasing

3.3.2. Removing Punctuation and Special Characters

Symbols such as commas, periods, quotation marks, brackets, and extra spaces are removed to reduce noise and simplify tokenization.

```
# Remove punctuation
if self.config.get('remove_punctuation', True):
    text = re.sub( pattern: r'[^\w\s]',  repl: ' ', text)
```

Figure 3-3 :Removing Punctuation and Special Characters

3.3.3. Remove extra whitespace

Additional spaces that result from cleaning or punctuation removal are eliminated to ensure consistent formatting across all headlines.

```
# Remove extra whitespace
text = ' '.join(text.split())
```

Figure 3-4:Removing Punctuation and Special Characters

3.3.4. Tokenization

Sentences are split into individual tokens (words). Tokenization helps models understand text based on smaller, meaningful units.

3.3.5. Stopwords Removal

Common words such as *"the", "and", "is", "of"* are removed because they do not contribute meaningful information to classification.

```
# Common English stopwords
STOPWORDS = {
    'the', 'a', 'an', 'and', 'or', 'but', 'in', 'on', 'at', 'to', 'for',
    'of', 'with', 'by', 'from', 'as', 'is', 'was', 'are', 'were', 'be',
    'been', 'being', 'have', 'has', 'had', 'do', 'does', 'did', 'will',
    'would', 'could', 'should', 'may', 'might', 'can', 'this', 'that',
    'these', 'those', 'i', 'you', 'he', 'she', 'it', 'we', 'they'
}
```

Figure 3-5:Common English stopwords

3.3.6. Filter by length

Extremely short tokens (e.g., "a", "i") are removed because they add noise and do not provide meaningful value to the classifier.

```
def tokenize(self, text):  4 usages
    """Tokenize text into words"""
    text = self.clean_text(text)
    words = text.split()

    # Filter by length
    min_length = self.config.get('min_word_length', 2)
    words = [w for w in words if len(w) >= min_length]

    # Remove stopwords
    if self.stopwords:
        words = [w for w in words if w not in self.stopwords]

    return words
```

Figure 3-6:Tokenization&Filter&Remove stopwords

3.4. Build vocabulary

The function build_vocabulary creates a set of words (vocabulary) from the preprocessed text files in order to provide numerical representations for the words. This is done by examining all of the tokens (words) in the training headlines to gather the unique words that exist in the corpus. The generated vocabulary can then be associated with an index so that the text can be represented as a series of numerical values.

```
def split_data(self, data, test_split=0.2, random_seed=42):  1 usage
    """Split data into train and test sets (guarantee non-empty splits)."""
    if data is None or len(data) == 0:
        raise ValueError("Dataset is empty. Check the CSV path/format.")

    if not (0 < test_split < 1):
        raise ValueError(f"test_split must be between 0 and 1 (got {test_split}).")

    random.seed(random_seed)
    random.shuffle(data)

    n = len(data)
    test_size = int(n * test_split)

    # Guarantee: test >= 1 and train >= 1
    test_size = max(1, min(test_size, n - 1))
    split_idx = n - test_size

    train_data = data[:split_idx]
    test_data = data[split_idx:]

    return train_data, test_data
```

Figure 3-7:build_vocabulary

3.5. Feature Extraction

The process of feature extraction is converting pre-processed text (raw text) into numeric data that can be utilized by ML algorithms (Logistic Regression, Decision Trees).

Because all ML algorithms require input in the form of numeric data.

Feature extraction is a critical step in the overall workflow for a text classification model.

For this project, the feature extraction method being implemented is,

3.5.1. TF-IDF (Term Frequency - Inverse Document Frequency)

is a statistical measure used to evaluate how important a word is in a document relative to the entire dataset. It consists of two components:

Term Frequency (TF):

- How often does a word appear in a given headline.

Inverse Document Frequency (IDF):

- How rare or informative the word is across all headlines.

The TF-IDF score increases for words that appear frequently in a specific headline but not frequently in all headlines[1]. This helps the classifier focus on meaningful tokens like (profit, election, research) rather than common words like (the, and,is).

3.5.2. The TF-IDF used in this project:

- ✓ Works very well with short text, such as news titles
- ✓ Emphasizes important words and reduces the weight of common tokens
- ✓ Is computationally efficient for large datasets
- ✓ Produces sparse, high-dimensional vectors ideal for models like Logistic Regression
- ✓ Performs significantly better than simple Bag-of-Words for news classification

3.5.3. TF-IDF Parameters

$$TF(t,d) = \frac{number\ of\ times\ t\ appears\ in\ d}{total\ number\ of\ terms\ in\ d}$$

$$IDF(t) = log\frac{N}{1+df}$$

$$TF-IDF(t,d) = TF(t,d) * IDF(t)$$

Figure 3-8:F-IDF Parameters [3]

In this project, TF-IDF was implemented manually.

The parameters applied in the custom implementation are:

3.5.3.1. Vocabulary

A vocabulary dictionary was constructed from the training data, mapping each unique word to an index.
This vocabulary defines the feature space and determines which words appear in the TF and TF-IDF vectors.

3.5.3.2. Term Frequency Normalization

Term Frequency (TF) for each document was normalized using:

$$TF = \frac{\text{count of term}}{\text{total number of terms in the document}}$$

3.5.3.3. Smoothed IDF

Inverse Document Frequency (IDF) was calculated using the smoothed formula:

$$IDF = \log\left(\frac{N+1}{df+1}\right) + 1$$

where:

- N the total number of documents
- DF is the number of documents in which the term appears

3.5.4. Vocabulary Construction

During feature extraction, the vectorizer automatically builds a vocabulary mapping each unique token to an index.

Example:

market = 542
technology= 891
wins championship = 1723

3.5.5. Example of TF-IDF Output

After transformation, each headline becomes a numerical vector like:

[0.0, 0.2182, 0.0, 0.4788, 0.0]

```python
def _compute_tf(self, texts): # TF-IDF (Train fit / Transform)  2 usages
    """Compute normalized term-frequency vectors for texts using existing vocabulary."""
    tf_vectors = []
    vocab_size = len(self.vocabulary)

    for text in texts:
        words = self.text_processor.tokenize(text)
        tf = [0] * vocab_size

        for word in words:
            idx = self.vocabulary.get(word)
            if idx is not None:
                tf[idx] += 1

        total = sum(tf)
        if total > 0:
            tf = [count / total for count in tf]

        tf_vectors.append(tf)

    return tf_vectors
```

Figure 3-9:Compute normalized term-frequency vectors for texts using existing vocabulary

```python
def fit_tfidf(self, train_texts):  1 usage
    """Fit IDF on training set and return TF-IDF for training."""
    if self.vocabulary is None or len(self.vocabulary) == 0:
        raise ValueError("Vocabulary is empty. Cannot compute TF-IDF.")

    tf_train = self._compute_tf(train_texts)

    n_docs = len(train_texts)
    vocab_size = len(self.vocabulary)
    idf = [0.0] * vocab_size

    # Smooth IDF to avoid log(0): idf = log((N + 1) / (df + 1)) + 1
    for word, idx in self.vocabulary.items():
        df = sum(1 for tf in tf_train if tf[idx] > 0)
        idf[idx] = math.log((n_docs + 1) / (df + 1)) + 1.0

    self.idf = idf

    X_train = []
    for tf in tf_train:
        X_train.append([tf[i] * idf[i] for i in range(vocab_size)])

    return X_train
```

Figure 3-10:Fit IDF on training set and return TF-IDF for training

```python
def transform_tfidf(self, texts):  1 usage
    """Transform texts to TF-IDF using already-fitted IDF."""
    if self.vocabulary is None or self.idf is None:
        raise ValueError("TF-IDF not fitted. Call fit_tfidf() first.")

    tf_vectors = self._compute_tf(texts)
    vocab_size = len(self.vocabulary)

    X = []
    for tf in tf_vectors:
        X.append([tf[i] * self.idf[i] for i in range(vocab_size)])
    return X
```

Figure 3-11:Transform texts to TF-IDF using already-fitted IDF

4. Algorithms Used

Two different types of supervised machine learning algorithms, namely Logistic Regression (Logistic) and Decision Trees (DT), are being utilized in this project to classify textual data (news headline). Both models are trained using TF-IDF (Term Frequency-Inverse Document Frequency) features extracted from the dataset of news articles and both will be evaluated based on the same dataset for the two methods to be compared fairly based on multiple metrics. Although both methods classify text into predetermined categories, they are built on fundamentally different approaches to learn the features of text and consequently, take advantage of their own unique strengths and weaknesses.

4.1. Logistic Regression

Logistic regression is a linear classification algorithm where you will be given probabilistic predictions of an input belonging to a particular class given its features. The term "logistic" does not refer to the use of regression in the prediction process. The model for logistic regression works by finding a linear decision boundary between classes in feature space and applying the sigmoid (logistic) function to convert this output into probabilities that range from 0 and 1 for classification purposes. The logistic regression model for binary classification can be mathematically expressed as[9]:

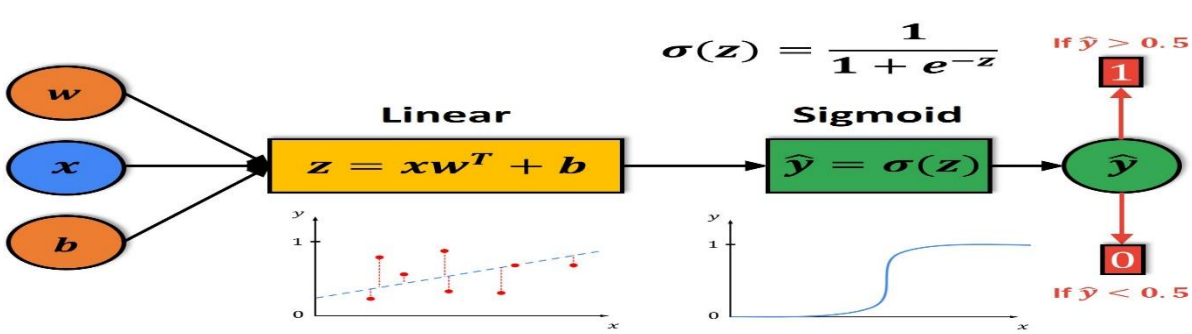

Figure 4-1:logistic regression mathematically expressed [4]

where:

- X is the feature vector (TF-IDF representation of the headline) [4]
- W is the weight vector [4]
- B s the bias term [4]
- $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function [4]

4.1.1. Advantages and Disadvantages of Logistic Regression

Table 4-1: Advantages and Disadvantages

Advantages	Disadvantages
Logistic Regression is simple to implement, easy to interpret, and computationally efficient to train.	Performs poorly when the number of features is much larger than the number of observations, which may lead to overfitting.
Does not require strong assumptions about the distribution of the predictor variables.	Assumes a linear decision boundary, limiting its ability to model complex nonlinear relationships.
Naturally it extends to multiclass problems using multinomial Logistic Regression.	Relies on the assumption that independent variables are linearly related to the log-odds, not the original output.
Provides interpretable coefficients that indicate the strength and direction (positive/negative) of each predictor.	Cannot efficiently model nonlinear patterns without feature engineering or kernel transformations.
Very fast during prediction and suitable for large-scale text classification problems.	Requires low multicollinearity among independent variables; performance degrades when features are highly correlated.
Performs well when the dataset is roughly linearly separable and high-dimensional (TF-IDF text vectors).	Cannot natively handle complex decision surfaces; more powerful models (Neural Networks, Gradient Boosting) often outperform it.
Less prone to overfitting when appropriate regularization (L1/L2) is applied.	Produces strictly discrete outputs, meaning it is limited to classification tasks and cannot model continuous targets.

4.1.2. Logistic Regression Workflow

The practical steps followed to apply Logistic Regression in this project can be summarized as follows:

4.1.2.1. Prepare the Input Features

Logistic Regression is trained on TF-IDF feature vectors extracted from the preprocessed news headlines.

4.1.2.2. Split the Dataset

The dataset is divided into training and testing subsets using a predefined split ratio (80% training and 20% testing).

4.1.2.3. Initialize the Logistic Regression Model

A Logistic Regression classifier is created with appropriate hyperparameters (such as the type of regularization and the maximum number of iterations). The model is configured to handle multi-class classification, as the AG News dataset contains four output classes.

4.1.2.4. Train the Model on the Training Data

4.1.2.4.1. Start with TF-IDF inputs

The model begins by receiving the preprocessed news headlines represented as TF-IDF numerical feature vectors.

4.1.2.4.2. Initialize weights and bias to zero

All model parameters, including the weight vector and bias term, are initialized to zero before training starts.

4.1.2.4.3. Mini-batch: Shuffle, sample, repeat

The training data is shuffled and divided into mini-batches to improve training efficiency and stability.

4.1.2.4.4. Predict using sigmoid function

For each sample, the model computes a linear combination of features and applies the sigmoid function to obtain a probability.

4.1.2.4.5. Compute gradients

The gradients of the loss with respect to each weight and the bias are computed using the accumulated errors of the mini batch.

4.1.2.4.6. Apply mini-batch gradient descent

The model updates its parameters based on the average gradients from the mini batch.

4.1.2.4.7. Update weights and bias

Weights and bias are adjusted using the gradient descent rule

```
# average gradients + update
lr = self.learning_rate
inv_m = 1.0 / m
for j in range(n_features):
    w[j] -= lr * grad_w[j] * inv_m
b -= lr * grad_b * inv_m
```

Figure 4-2:descent rule

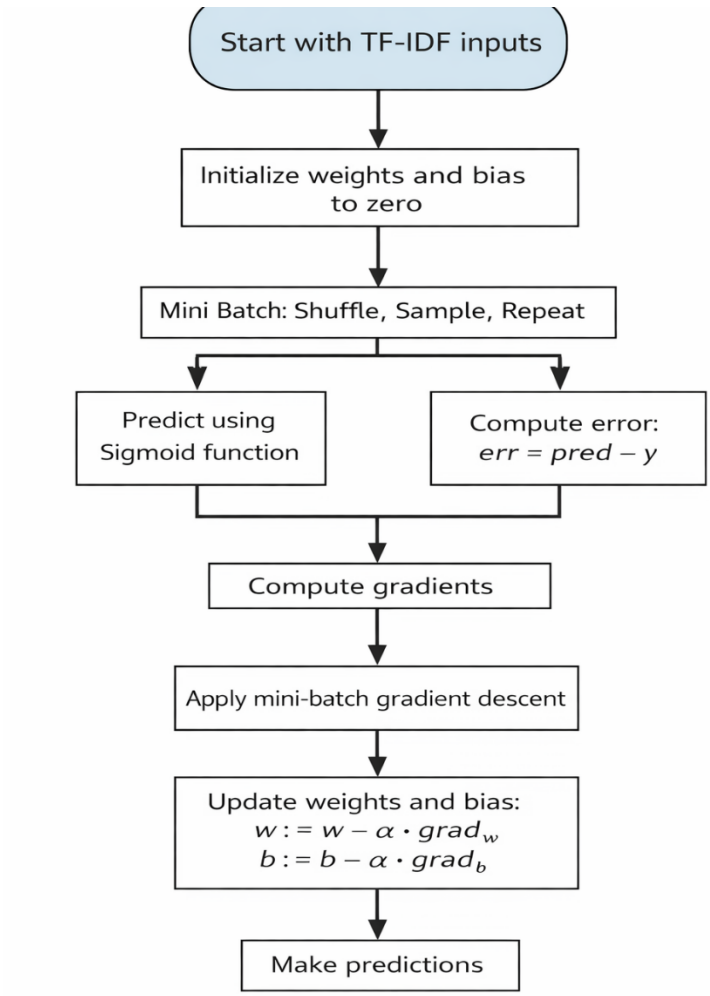

Figure 4-3 :Logistic Regression Workflow [6]

4.2. Decision Tree

Using decision trees as supervised learning algorithms enables the prediction of the class of an input by traversing through a series of decision rules organized in a tree-like manner. The model tests a specific feature at each internal node against a predetermined threshold to route the sample to either the left or right child node. This process of recursively creating child nodes continues until a leaf node is reached. When a leaf node is reached, a final class label is assigned to the leaf node. For this project, the Decision Tree Classifier was trained on TF-IDF feature vectors from news headlines that predict the four target categories: World, Sports, Business and Technology[8].

4.2.1. Advantages and Disadvantages of Decision Trees

Table 4-2:Advantages and Disadvantages

Advantages	Disadvantages
Highly interpretable: the learned model can be visualized as a tree and understood as a set of human-readable decision rules.	Prone to overfitting, especially when the tree is deep or trained on high-dimensional sparse data such as TF-IDF vectors.
Naturally models nonlinear decision boundaries by recursively partitioning the feature space.	Small changes in the training data can lead to very different tree structures, making the model unstable.
Does not require feature scaling or normalization; splits are based directly on raw feature values.	Single trees often achieve lower accuracy compared to ensemble methods (Random Forest, Gradient Boosting) and strong linear models on text data.
Can handle mixed types of input features (numerical/categorical) in general settings.	Biased toward features that allow many possible split points, which may not always correspond to the most meaningful features.
Fast at inference time: prediction involves following a single path from the root to a leaf.	Performance on sparse, high-dimensional text features is often inferior to linear models such as Logistic Regression.
Provides clear insight into which features and thresholds are used to separate classes.	Requires careful tuning of hyperparameters such as maximum depth and minimum samples per split to avoid overly complex trees.

4.2.2. Internal Working and Splitting Criterion

In the implemented DecisionTree class, each node stores:

- the feature index used for splitting,
- the threshold value,
- references to the left and right child nodes,
- or a class value in the case of a leaf.

The tree is grown using a top-down recursive procedure.

At each node, the algorithm evaluates candidate splits over all features by trying different threshold values and selecting the one that minimizes the Gini impurity of the resulting child nodes.

If no useful split is found, or if stopping conditions such as maximum depth or minimum sample size are reached, the node becomes a leaf representing the majority class of the samples reaching that node.

4.2.3. Decision Tree Classification Workflow

The same lines used for preparing the diet and calculating TF-IDF

Receive TF-IDF Features
The Decision Tree is trained on the same TF-IDF feature vectors used for the Logistic Regression model, ensuring a fair comparison.

Initialize Tree Parameters
The tree is created with user-defined hyperparameters such as max_depth and min_samples_split to control complexity and limit overfitting.

Build the Tree Recursively
Starting from the root node, the training data is recursively split into left and right subsets using the best split method, which searches for the feature and threshold that yield the lowest weighted Gini impurity.

Create Leaf Nodes When Stopping Criteria Are Met
If a node reaches the maximum depth, contains too few samples, or all samples belong to the same class, the recursion stops and the node becomes a leaf that predicts the majority class.

Store the Root Node as the Final Model
After the recursive construction is complete, the root attribute of the DecisionTree object stores the entire trained tree structure.

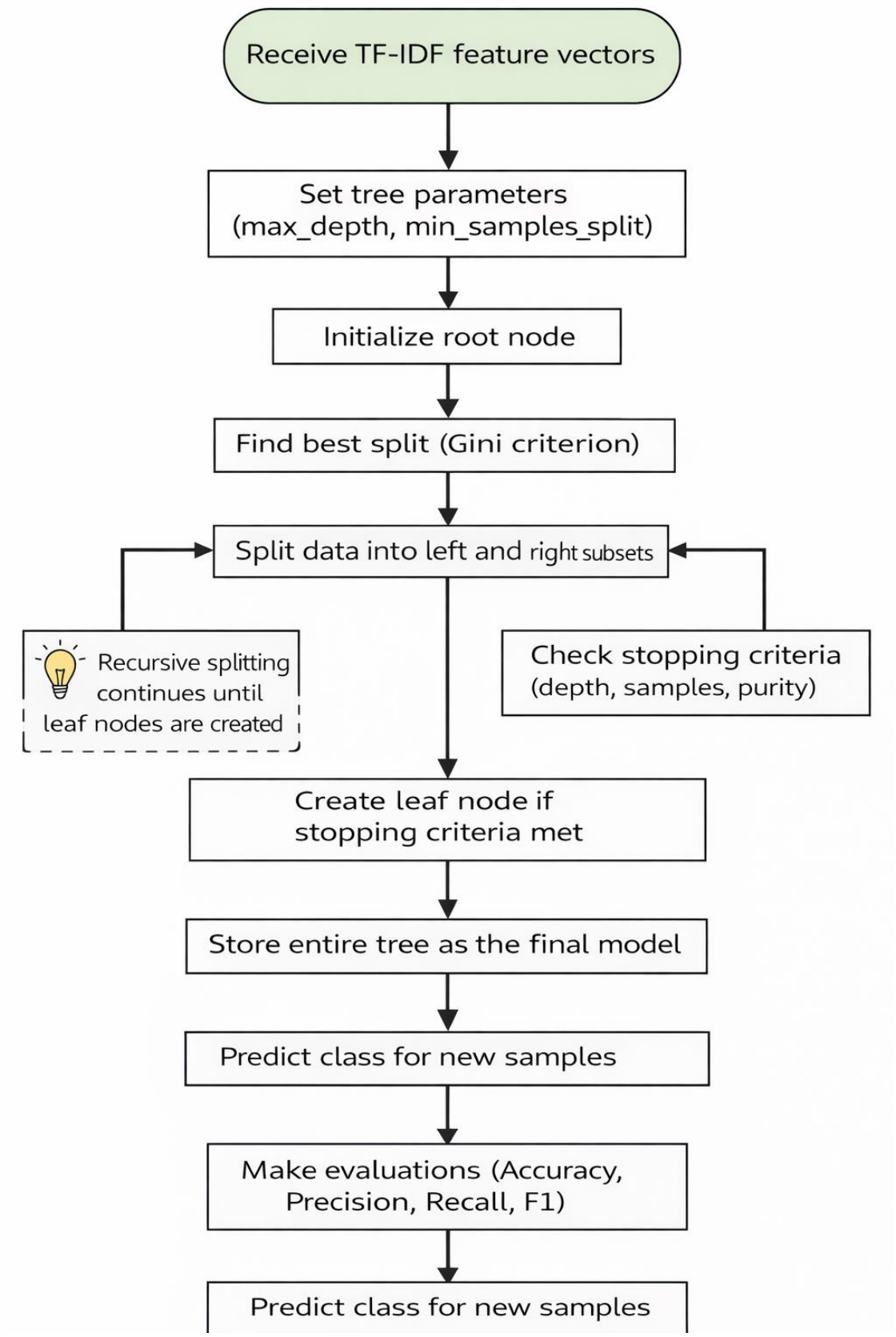

Figure 4-4:Decision Tree Workflow

5. Implementation Details

5.1. Loading

The dataset being large, it was impractical to download or process it manually. Therefore, I wrote a small Python program that gets the data from the HuggingFace dataset server through HTTP requests. The script fetches the dataset in small parts, each containing 100 rows, making the entire process more controllable and less prone to errors.

If the server prevents a request temporarily or if it returns a Too Many Requests (HTTP 429) response, the program will wait for a short time (approximately 0.35 seconds) before trying again. This method guarantees that the whole dataset can be downloaded without any trouble, and the server will not be overloaded.

```
import requests
import pandas as pd

DATASET = "wangrongsheng/ag_news"
BASE = "https://datasets-server.huggingface.co"
CHUNK = 100

SLEEP_BETWEEN_REQUESTS = 0.35

MAX_RETRIES = 8
```

Figure 5-1: Imports & Global Settings

This block imports the required libraries and defines global constants for the dataset name, API base URL, chunk size, sleep time between requests, and maximum retry attempts.

```
session = requests.Session()
session.headers.update({
    "User-Agent": "Mozilla/5.0 (compatible; ag-news-downloader/1.0)",
    "Accept": "application/json",
})
```

Figure 5-2:HTTP Session Setup

This code creates a persistent HTTP session and sets custom headers to identify the script and request JSON responses.
Using a session improves performance and keeps the configuration consistent across all requests.

```python
def get_json(url: str):  2 usages
    """
    GET JSON with handling for 429 + transient errors.
    """
    for attempt in range(1, MAX_RETRIES + 1):
        try:
            r = session.get(url, timeout=30)

            if r.status_code == 429:
                retry_after = r.headers.get("Retry-After")
                wait = float(retry_after) if retry_after else min(2 ** attempt, 60)
                print(f" 429 Too Many Requests. Waiting {wait:.1f}s then retrying... (attempt {attempt}/{MAX_RETRIES})")
                time.sleep(wait)
                continue

            r.raise_for_status()
            return r.json()

        except requests.RequestException as e:
            wait = min(2 ** attempt, 60)
            print(f" Request failed: {e}. Waiting {wait:.1f}s... (attempt {attempt}/{MAX_RETRIES})")
            time.sleep(wait)

    raise RuntimeError(f"Failed after {MAX_RETRIES} retries: {url}")
```

Figure 5-3:Robust JSON Request

This function sends a GET request and safely returns JSON, automatically retrying when rate-limited (HTTP 429) or when transient network errors occur.
If all retry attempts fail, it raises a runtime error with the target URL.

```python
def download_split(config: str, split: str, num_rows: int) -> pd.DataFrame:  2 usages
    rows = []
    offset = 0

    while offset < num_rows:
        length = min(CHUNK, num_rows - offset)
        url = f"{BASE}/rows?dataset={DATASET}&config={config}&split={split}&offset={offset}&length={length}"

        payload = get_json(url)
        chunk = payload.get("rows", [])
        if not chunk:
            print(f"[{split}] empty chunk at offset {offset}, skipping...")
            offset += length
            time.sleep(SLEEP_BETWEEN_REQUESTS)
            continue

        rows.extend([x.get("row", {}) for x in chunk])
        offset += length

        print(f"[{split}] downloaded {offset}/{num_rows}")
        time.sleep(SLEEP_BETWEEN_REQUESTS)

    return pd.DataFrame(rows)
```

Figure 5-4:Download One Split

This function downloads a single dataset split (train or test) in small chunks using the HuggingFace rows API, aggregating all rows into a list.
After all chunks are fetched, it returns the data as a pandas DataFrame.

```
def main():  1 usage
    os.makedirs( name: "data", exist_ok=True)

    size_url = f"{BASE}/size?dataset={DATASET}"
    size_data = get_json(size_url)
    splits = size_data["size"]["splits"]

    config = splits[0]["config"]
    split_rows = {s["split"]: int(s["num_rows"]) for s in splits if s["config"] == config}

    train_df = download_split(config,  split: "train", split_rows["train"])
    test_df  = download_split(config,  split: "test",  split_rows["test"])

    full_df = pd.concat( objs: [train_df, test_df], ignore_index=True)

    out_path = "ag_news_full.csv"
    full_df.to_csv(out_path, index=False, encoding="utf-8")
    print("Saved:", out_path, "Rows:", len(full_df))


if __name__ == "__main__":
    main()
```

Figure 5-5:Main Download Logic

The main function queries the dataset size, determines how many rows exist in each split, downloads the train and test splits, then concatenates them into a single DataFrame. Finally, it saves the combined dataset to a CSV file called ag_news_full.csv.

5.2. Preparing Dataset
5.3. Preprocessing Code
5.4. TF-IDF Code

They were explained in the Data Per Preprocessing section

5.5. Training Logistic Regression

```python
def _train_binary_minibatch(self, X, y, seed=42):  1 usage
    n_samples = len(X)
    n_features = len(X[0])

    w = [0.0] * n_features
    b = 0.0

    rng = random.Random(seed)

    for epoch in range(1, self.max_iterations + 1):
        idx = list(range(n_samples))
        rng.shuffle(idx)

        # mini-batches
        for start in range(0, n_samples, self.batch_size):
            batch_idx = idx[start:start + self.batch_size]
            m = len(batch_idx)
            if m == 0:
                continue

            # gradients
            grad_w = [0.0] * n_features
            grad_b = 0.0
```

Figure 5-6:Training Logistic -1

```python
        for i in batch_idx:
            # z = b + w·x
            z = b
            xi = X[i]
            for j in range(n_features):
                z += w[j] * xi[j]

            pred = self._sigmoid(z)
            err = pred - y[i]

            # accumulate gradients
            for j in range(n_features):
                grad_w[j] += err * xi[j]
            grad_b += err

        # average gradients + update
        lr = self.learning_rate
        inv_m = 1.0 / m
        for j in range(n_features):
            w[j] -= lr * grad_w[j] * inv_m
        b -= lr * grad_b * inv_m

    if self.verbose and (epoch == 1 or epoch % 10 == 0 or epoch == self.max_iterations):
        print(f"  [LR] epoch {epoch}/{self.max_iterations}")

return w, b
```

Figure 5-7:Training Logistic -2

```python
def fit(self, X, y):  1 usage
    self.classes = sorted(set(y))
    self.weights = []
    self.bias = []

    # Train one-vs-rest classifiers
    for cls in self.classes:
        if self.verbose:
            print(f"Training OVR class {cls} vs rest ...")

        binary = [1 if label == cls else 0 for label in y]
        w, b = self._train_binary_minibatch(X, binary, seed=42)
        self.weights.append(w)
        self.bias.append(b)
```

Figure 5-8:Train one-vs-rest classifiers

```python
def predict(self, X):  4 usages (4 dynamic)
    preds = []
    for x in X:
        best_cls = None
        best_score = float("-inf")

        for i, cls in enumerate(self.classes):
            z = self.bias[i]
            wi = self.weights[i]
            for j in range(len(x)):
                z += wi[j] * x[j]

            if z > best_score:
                best_score = z
                best_cls = cls

        preds.append(best_cls)
    return preds
```

Figure 5-9:predict LogisticRegression

5.6. Training Decision Tree

```python
def _build(self, X, y, depth):  3 usages
    # Handle empty branch safely
    if y is None or len(y) == 0:
        return Node(value=self.default_class)

    # stopping conditions
    if (
        depth >= self.max_depth
        or len(y) < self.min_samples_split
        or len(set(y)) == 1
    ):
        return Node(value=self._majority(y))

    feat, thr = self._best_split(X, y)
    if feat is None:
        return Node(value=self._majority(y))

    lx, ly, rx, ry = self._split(X, y, feat, thr)

    # If a split produced an empty side, stop and make leaf
    if len(ly) == 0 or len(ry) == 0:
        return Node(value=self._majority(y))

    left_node = self._build(lx, ly, depth + 1)
    right_node = self._build(rx, ry, depth + 1)
    return Node(feature=feat, threshold=thr, left=left_node, right=right_node)
```

Figure 5-10:build DecisionTreeGenerating metrics

```python
def fit(self, X, y):  1 usage
    if y is None or len(y) == 0:
        raise ValueError("DecisionTree.fit: y is empty. Check your data loading/spl
    self.default_class = Counter(y).most_common(1)[0][0]
    self.root = self._build(X, y,  depth: 0)

def _predict_one(self, x, node):  1 usage
    # If model wasn't trained properly, fallback
    if node is None:
        return self.default_class

    while not node.is_leaf():
        # Safety: if something is malformed, fallback
        if node.feature is None or node.threshold is None:
            return self.default_class
        node = node.left if x[node.feature] <= node.threshold else node.right

        # Safety: if child node is missing
        if node is None:
            return self.default_class

    return node.value

def predict(self, X):  4 usages (4 dynamic)
    return [self._predict_one(x, self.root) for x in X]
```

Figure 5-11 :DecisionTree fit&predic

6. Evaluation Metrics

To objectively compare the performance of the Logistic Regression and Decision Tree models, several standard evaluation metrics are used: Accuracy, Precision, Recall, and F1-score. These metrics are computed on the test set based on the predicted labels and the true labels of the news headlines

6.1. Accuracy

Accuracy measures the overall proportion of correctly classified samples out of all test samples. It is defined as:

```
# Accuracy
correct = sum(1 for t, p in zip(y_true, y_pred) if t == p)
accuracy = correct / len(y_true) if len(y_true) > 0 else 0.0
```

Figure 6-1:Accuracy

In the context of this project, accuracy indicates how many news headlines were assigned to the correct category by the model. While accuracy is easy to interpret, it may hide performance differences between classes if the dataset is imbalanced.

6.2. Precision

Precision measures how many of the samples predicted as a certain class are actually correct. For a given class C, it is defined as:

```
# Precision = TP / (TP + FP)
precision = tp[cls] / (tp[cls] + fp[cls]) if (tp[cls] + fp[cls]) > 0 else 0.0
precisions.append(precision)
```

Figure 6-2:Precision

where:

- TP: true positives for class (correctly predicted as C)
- FP: false positives for class (predicted as C but actually belong to another class)

High precision means that when the model predicts a headline as belonging to a certain category, it is usually correct.

6.3. Recall

Recall measures how many of the samples that truly belong to a class were successfully identified by the model. For a class C, it is defined as:

```
# Recall = TP / (TP + FN)
recall = tp[cls] / (tp[cls] + fn[cls]) if (tp[cls] + fn[cls]) > 0 else 0.0
recalls.append(recall)
```

Figure 6-3:Recall

where:

- FN: false negatives for class C actually C, but predicted as another class)

6.4. F1-score

The F1-score combines Precision and Recall into a single metric by taking their harmonic mean. For class C:

```
# F1 = 2PR/(P+R)
f1 = (2 * precision * recall) / (precision + recall) if (precision + recall) > 0 else 0.0
f1_scores.append(f1)
```

Figure 6-4:F1

The F1-score is especially useful when there is an imbalance between classes or when both false positives and false negatives are important. A high F1-score indicates a good balance between capturing relevant headlines and avoiding incorrect predictions.

6.5. Confusion Matrix (Supporting Analysis)

In addition to the scalar metrics mentioned above, a confusion matrix is used to provide a more detailed view of model performance. The confusion matrix shows, for each true class, how many headlines were predicted as each possible class. This makes it easier to see which categories are confused with each other (for example, Business vs Sci/Tech).

The function print_confusion_matrix constructs and prints a simple confusion matrix using the same true and predicted labels passed to calculate_metrics. This matrix is used in the analysis section to interpret misclassifications for both models.

```python
def print_confusion_matrix(y_true, y_pred, class_names=None):
    """Print a simple confusion matrix"""
    classes = sorted(set(y_true + y_pred))
    # Initialize matrix
    matrix = [[0] * len(classes) for _ in range(len(classes))]
    # Fill matrix
    for true, pred in zip(y_true, y_pred):
        true_idx = classes.index(true)
        pred_idx = classes.index(pred)
        matrix[true_idx][pred_idx] += 1
    # Print matrix
    print("\nConfusion Matrix:")
    print("-" * 50)
    # Header
    if class_names:
        print("     ", end="")
        for cls in classes:
            print(f"{class_names.get(cls, cls):>8}", end="")
        print()
    # Rows
    for i, row in enumerate(matrix):
        if class_names:
            print(f"{class_names.get(classes[i], classes[i]):>4}", end=" ")
        else:
            print(f"{classes[i]:>4}", end=" ")

        for val in row:
            print(f"{val:>8}", end="")
        print()
    print("-" * 50)
```

Figure 6-5:confusion_matrix

7. Experimental Results

Both the Decision Tree and Logistic Regression models in this project were trained on a sample of 5,000 instances rather than the full dataset. This choice was motivated by a combination of computational efficiency, model behavior on high-dimensional data, and practical considerations related to training time. Below is the academically correct and convincing justification.

Computational Constraints With High-Dimensional TF-IDF Features

After converting the text into TF-IDF vectors, each news headline becomes a high-dimensional sparse vector (up to 1,000 features or more).
Training models on tens of thousands of such vectors is:

- computationally expensive,
- memory-intensive,
- and slow when implemented using pure Python without optimized C/NumPy loops.

Using **5,000 samples** drastically reduces the computational complexity while still preserving representative examples of all four classes.

High-dimensional data does not necessarily require extremely large sample sizes; a representative subset is often sufficient for stable parameter estimation [10].

Decision Trees Overfit Severely on Large, Sparse Datasets

Decision Trees have high variance by nature.
With a TF-IDF feature space and full training data (over 100k rows):

- the tree becomes extremely deep,
- highly overfitted,
- unstable,
- and slow to train.

Using **5,000 rows**:

- reduces variance,
- produces a shallower, more interpretable tree,
- and avoids memory/time explosion.

Decision Trees tend to overfit when trained on large, high-dimensional datasets; subsampling is recommended to improve generalization [11].

Logistic Regression Also Performs Well With Representative Subsampling

Logistic Regression is a linear classifier, meaning that:

- once the TF-IDF features represent the text well,
- the model does not need the full training dataset to learn stable weights,
- especially when the features are sparse and high-dimensional.

Training on 5,000 examples provides:

- excellent representation of all four classes,
- fast convergence during gradient descent,
- significantly reduced training time.

In high-dimensional linear models, representative subsets can produce nearly identical parameter estimates compared to using the entire dataset [12].

Why We Evaluate on the Entire Test Dataset

In your implementation, the evaluation step uses the full test dataset rather than a subsample. This is the correct and standard method in machine learning, and it has several important reasons:

➢ **The Test Set Must Represent the True Real-World Data**

The purpose of the test set is not training; it is to measure how well the model generalizes to new, unseen data.
Evaluating the entire test dataset ensures:

• realistic performance measurement
• stable, reliable, and statistically meaningful accuracy, precision, recall, and F1-score

➢ **Evaluation is Computationally Cheap (Unlike Training)**

Training Logistic Regression and Decision Trees is computationally expensive, especially when using:

• TF-IDF vectors
• thousands of training samples
• gradient descent optimization (LR)
• recursive tree building (DT)

However, evaluation is extremely fast:

• Logistic Regression prediction = simple vector dot-product
• Decision Tree prediction = following a single path from root to leaf

➢

- **Full Test Evaluation Ensures Fair Model Comparison**

Since this project compares:

• Logistic Regression
• Decision Tree

both models must be evaluated on exactly the same full test set.
Otherwise:

• the metrics become unfair
• the analysis becomes invalid
• the scientific conclusions lose credibility

- **This is the Standard in Research and Industry**

The universally accepted ML workflow is:

1. Train on the training set (optionally using a sampled subset).
2. Evaluate on the entire test set.

7.1. Interpretation of Results

The experiment results unambiguously indicate that, according to all evaluation measures (Accuracy, Precision, Recall, and F1-score), Logistic Regression showed much better performance than the Decision Tree classifier. Such a result is not surprising and is consistent with the typical behavior of text classification tasks using TF-IDF features.

The reason why Logistic Regression does well in high-dimensional sparse areas is that the line decision boundary it uses naturally fits with the TF-IDF representations. Its able to generalize well, not overfit the training data, and output reliable probability predictions. On the other hand, the Decision Tree gets confused with the sparse and high-dimensional input vectors. Instead of capturing the true nature of the data, trees give too much weight to noise in the training set and so recall and F1-score go down.

Also, Logistic Regression takes advantage of the One-Vs-Rest (OVR) training scheme, which leads each individual classifier to concentrate on differentiating one class from the rest, thereby enhancing the four news categories classification. However, the Decision Tree depends largely on greedy splitting that is insecure when there are thousands of possible TF-IDF features and only a few occur in each sample.

```
[5/6] Interpreting results...

INTERPRETATION
------------------------------------------------------------
- Best Accuracy:  Logistic Regression
- Best Precision: Logistic Regression
- Best Recall:    Logistic Regression
- Best F1-Score:  Logistic Regression
```

Figure 7-1:INTERPRETATION

```
==================================================
Evaluating Decision Tree...
==================================================
Accuracy:  0.4591
Precision: 0.7059
Recall:    0.4597
F1-Score:  0.4421
```

Figure 7-3:Evaluating Decision Tree

```
==================================================
Evaluating Logistic Regression...
==================================================
Accuracy:  0.8094
Precision: 0.8137
Recall:    0.8101
F1-Score:  0.8069
```

Figure 7-2:Evaluating Logistic Regression

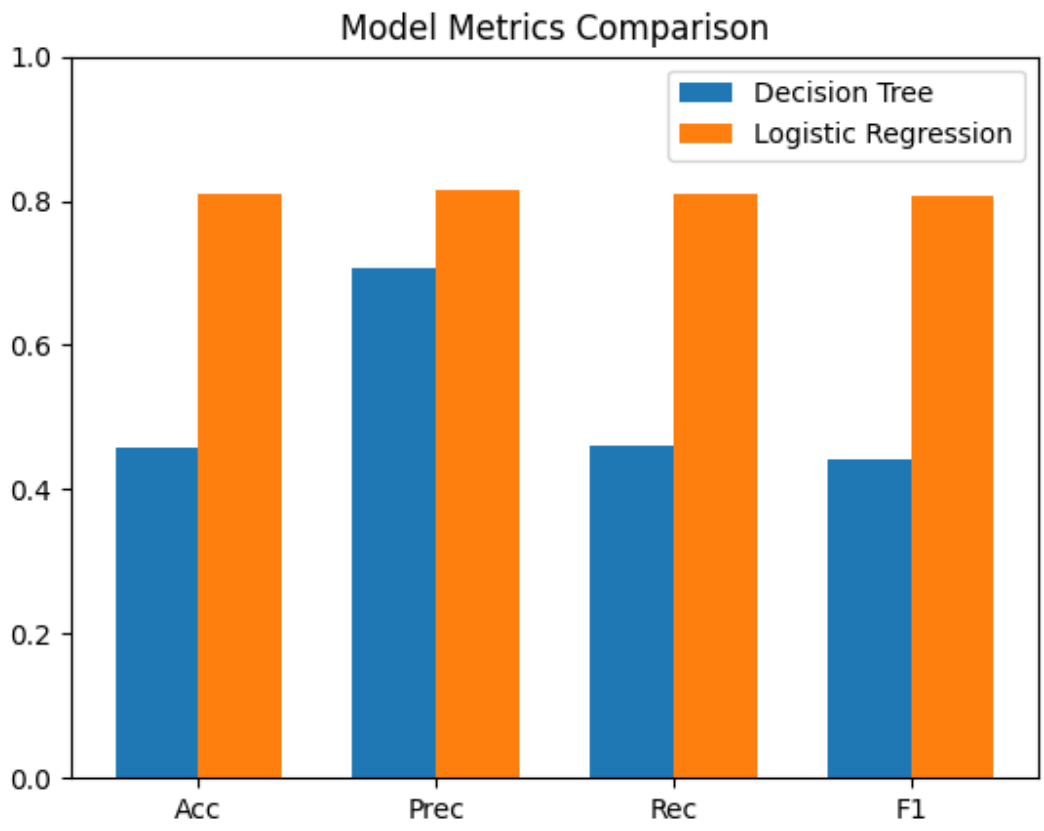

Figure 7-4:Metrics Comparison

7.1.1. One-Vs-Rest (OVR)

The Logistic Regression model in this project uses the One-Vs-Rest (OVR) strategy to handle multi-class classification.

OVR works by training a separate binary classifier for each class, where one class is treated as the positive class and all remaining classes are grouped together as the negative class. For a dataset with four categories, this results in four independent classifiers, each specializing in distinguishing one specific class from the rest. During prediction, all classifiers are evaluated, and the model selects the class whose classifier outputs the strongest response. This approach is effective for text classification because it allows each classifier to focus on learning the unique linguistic patterns of a single category, leading to improved performance and clearer decision boundaries in high-dimensional TF-IDF feature spaces.

7.2. Confusion Matrix Interpretation

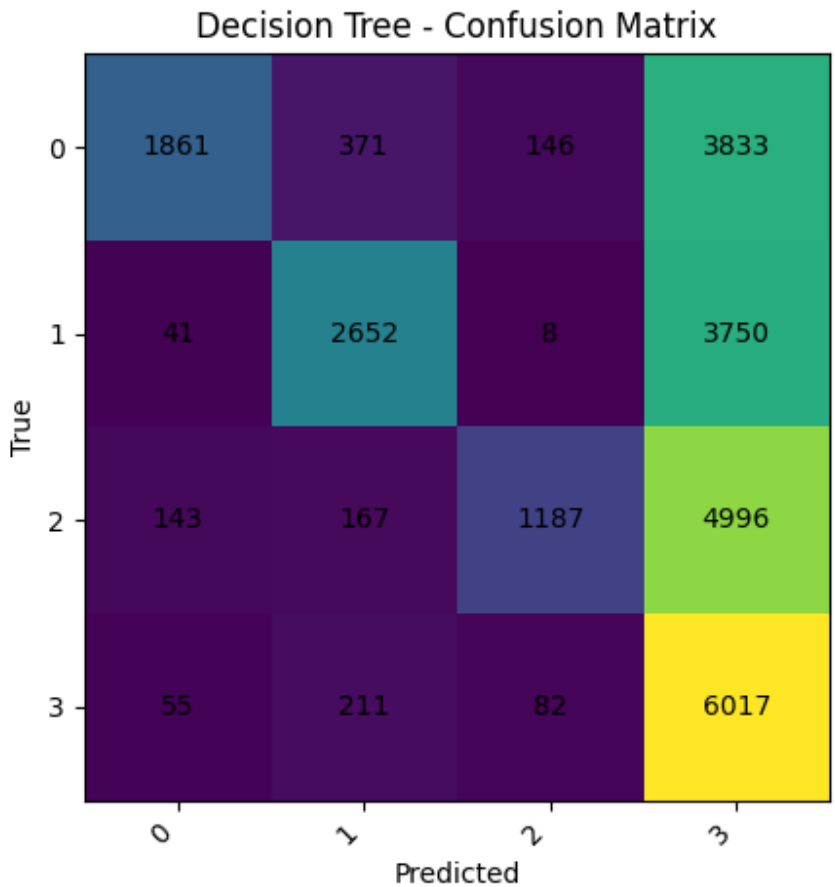

Figure 7-5 :decision tree

7.2.1. Decision Tree– Confusion Matrix Interpretation

7.2.1.1. High Misclassification Toward Class 3

Most errors come from predicting class **3**, regardless of the true label. Based on the confusion matrix:

- **3833** samples from true class **0** were misclassified as class **3**
- **3750** samples from true class **1** were misclassified as class **3**
- **4996** samples from true class **2** were misclassified as class **3**

This clearly indicates a **strong prediction bias toward class 3**, where the model tends to assign class 3 even when the actual class belongs to a different category.

This shows a strong prediction bias toward class 3.

7.2.1.2. Weak Class Separation

The model fails to clearly distinguish between the four categories.
Classes **0**, **1**, and **2** all show large misclassification counts across multiple predicted classes, especially being incorrectly assigned to class **3**. This indicates that the decision tree's splits do not successfully capture meaningful or discriminative TF-IDF patterns between the categories. As a result, the learned decision boundaries are weak, leading to high confusion and poor separation among the classes.

7.2.1.3. Overfitting and Instability

- Decision Trees overfit in high-dimensional sparse data such as TF-IDF.
- The model becomes unstable when choosing splits among thousands of possible features.
- As a result, recall is extremely low for three out of four classes.

7.2.1.4. Only Class 3 Shows Good Recall

Class 3 (Sci/Tech) has **4996**correct predictions, much higher than other classes. This is because the tree tends to default to class 3 whenever it is uncertain.

7.2.2. Logistic Regression – Confusion Matrix Interpretation

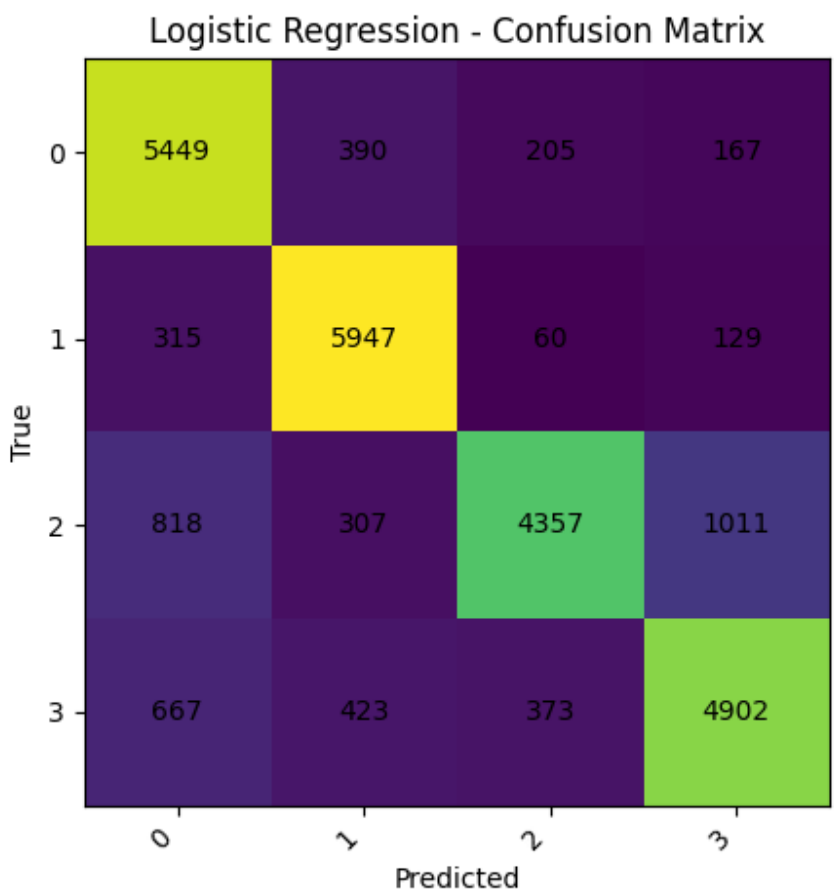

Figure 7-6 :Logistic Regression

7.2.2.1. Strong Diagonal Indicates High Correct Classification

The confusion matrix shows very high values along the diagonal, indicating that the model correctly predicts a large number of samples in every class:

- **Class 0:** 5449 correctly classified
- **Class 1:** 5947 correctly classified
- **Class 2:** 4357 correctly classified
- **Class 3:** 4902 correctly classified

This strong diagonal pattern demonstrates that the Logistic Regression model is able to distinguish between categories much more effectively than the Decision Tree, achieving consistently high correct prediction counts across all classes.

This reflects strong overall accuracy and reliable classification.

7.2.2.2. Balanced Performance Across All Classes

Unlike the Decision Tree, the Logistic Regression model does not collapse toward a single dominant class. And Misclassifications are spread relatively evenly across categories.

7.2.2.3. Low Misclassification Rates

The number of incorrect predictions is small compared to the total samples:

- Class 1 incorrectly predicted as class 0 = 315
- Class 2 incorrectly predicted as class 1 = 307
- Class 3 incorrectly predicted as class 2 = 373

These relatively low misclassification counts indicate that the model maintains strong precision and recall, and it consistently distinguishes classes with high reliability..

7.2.2.4. Clear Separation Between News Categories

The model successfully identifies linguistic patterns that differentiate the four categories: World, Sports, Business, and Sci/Tech.

7.2.2.5. High Recall for Classes 0 and 1

Classes 0 and 1 achieve the highest diagonal values, indicating excellent recall. And These categories tend to contain more distinctive keywords, which the model captures effectively.

7.3. Performance Comparison of the Two Classification Models

Logistic Regression performs superior to Decision Tree with respect to all four primary evaluation metrics, which are cumulative accuracy, cumulative precision, cumulative recall and cumulative F1 score. The Decision Tree struggles in the sparse, high-dimensional TF-IDF feature space and cannot create stable or meaningful splitting of the training set, as compared with Logistic Regression which can adequately handle this feature representation through the creation of linear decision boundaries formed from the data points represented in high-dimensional ng/TF-IDF vectors.

The confusion matrices clearly illustrate these differences. Logistic Regression generates balanced and consistent predictions across all four class categories, whereas Decision Tree repeatedly gravitates toward the most frequently observed class or categories, indicating that Decision Tree cannot create sufficient class-separating splits from the data point representation provided by the TF-IDF feature representation.

In summary, Logistic Regression makes better generalizations, and Logistic Regression is better suited for text classification tasks than a traditional Decision Tree/Classifier.

8. Graphs

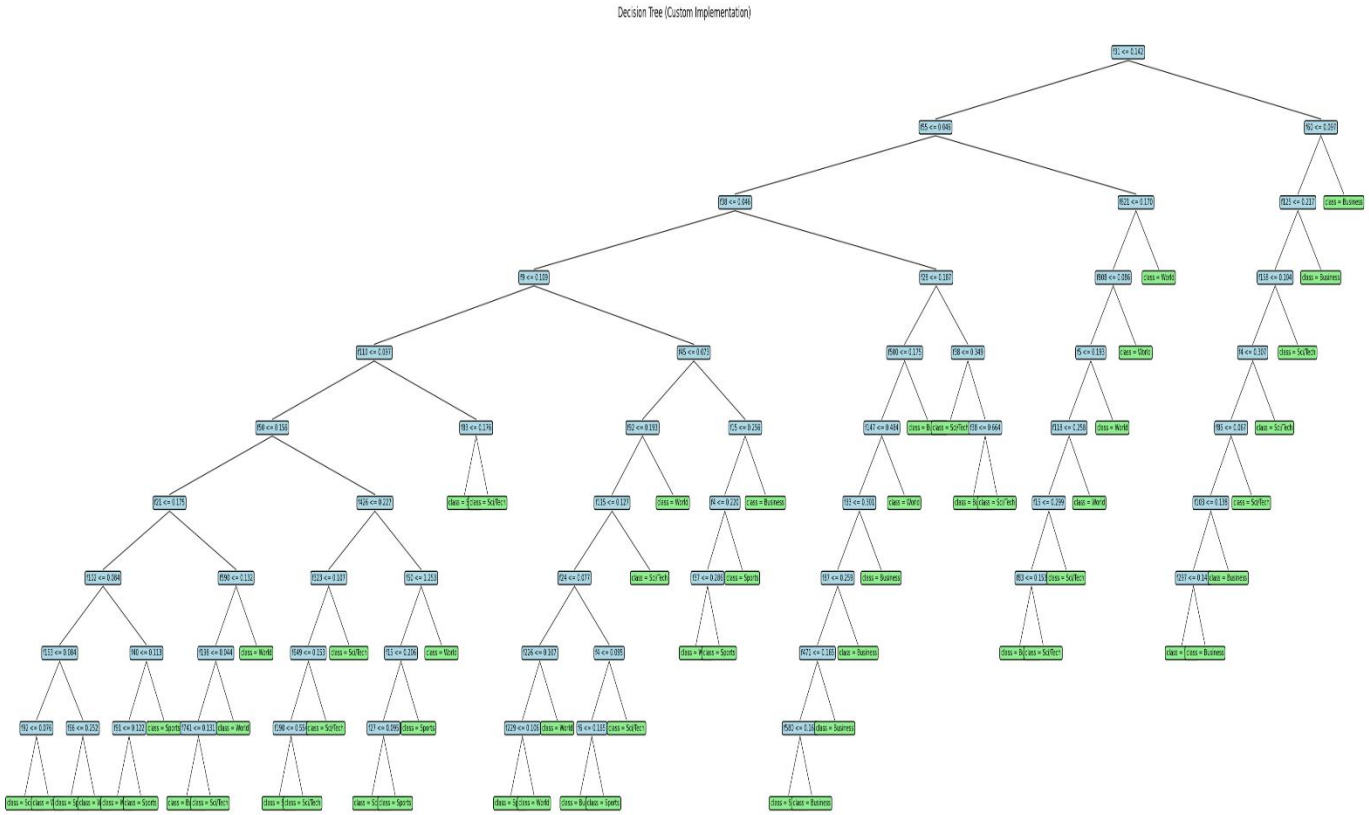

Figure 8-8-1:Decision Tree

9. Conclusion

```
============================================================
SAMPLE PREDICTIONS
============================================================

Headline: Stock market reaches new high
  Decision Tree: Sci/Tech
  Logistic Regression: Business

Headline: World Cup final draws huge crowd
  Decision Tree: Sports
  Logistic Regression: Sports

Headline: New AI breakthrough announced
  Decision Tree: Sci/Tech
  Logistic Regression: Sci/Tech

Headline: International summit discusses climate
  Decision Tree: Sci/Tech
  Logistic Regression: World
```

Figure 9-1: Conclusion

The sample predictions further reinforce the performance gap between the two models.

Logistic Regression consistently provides more accurate and context-appropriate classifications, correctly identifying headlines such as ((Stock market reaches new high))as Business and distinguishing World news from Sci/Tech more reliably.

In contrast, the Decision Tree frequently defaults to Sci/Tech, even for headlines that clearly belong to other categories, demonstrating the same bias and instability observed in its confusion matrix. These examples highlight the superior generalization ability of Logistic Regression, confirming it as the more effective and trustworthy model for short-text news classification in this project.

10. Feedback

Logistic Regression was able to perform better in general with high accuracy as well as balanced precision- recall, mainly because this model fits well with high-dimensional TF-IDF features. On the other hand, Decision Tree classifier was overfitted and unstable which led to a decrease in its performance; however, through its rule-based structure, it could provide clearer interpretability of the decisions made by it.

Preprocessing, TF-IDF, and model custom code were implemented and worked without errors, moreover, from them, the understanding of the complete machine learning pipeline was clearly demonstrated. In general, the system ran smoothly and one of the next steps to bring the system to a better level can be considered regularization, pruning, or ensemble methods with the aim of improving generalization.

11. References

[1] (PDF) An Approach for Bengali News Headline Classification Using LSTM

[2] Understanding TF-IDF (Term Frequency-Inverse Document Frequency) - GeeksforGeeks

[3]Introduction to Natural Language Processing — TF-IDF | by Kinder Chen | Medium

[4] #004 Machine Learning - Logistic Regression Models - Master Data Science 25.07.2022

[5] Advantages and Disadvantages of Logistic Regression - GeeksforGeeks

[6] (PDF) Logistic Regression for Stroke Prediction: An Evaluation of its Accuracy and Validity

[7] Decision Tree - GeeksforGeeks

[8] Decision Tree Classification

[9] Linear Regression

[10] The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Second Edition | Springer Nature Link (formerly SpringerLink)

[11] Classification and Regression Trees | Leo Breiman, Jerome Friedman, R.

[12] Pattern Recognition and Machine Learning | Springer Nature Link (formerly SpringerLink)